

OrthoAnalyse

Analyse céphalométrique assistée par IA

Application web pour l'automatisation de l'analyse céphalométrique en
orthodontie

Sahnoun

Concepteur Développeur d'Applications

Certification RNCP31678

Mai 2026

1. Présentation du projet	3
2. Contexte médical & Problématique	4
3. Stack technique	6
4. Architecture MVC	7
5. Base de données	8
6. Diagramme de classes UML	9
7. Diagramme de cas d'utilisation	10
8. Maquettage de l'application	11
9. Authentification & JWT	13
10. RBAC — Contrôle d'accès	14
11. Canvas interactif	15
12. Moteur de mesures	16
13. Analyse IA — HRNet	17
14. Génération de rapports	18
15. Tests et qualité	19
16. Sécurité	20
17. Problématiques rencontrées	21
18. Veille technologique	22
19. Conclusion & Perspectives	23

1. Présentation du projet

Qu'est-ce que OrthoAnalyse ?

Application web complète d'**analyse céphalométrique** destinée aux cabinets d'orthodontie.

- Détection automatique de **32 landmarks** crâniens et faciaux par IA
- Calcul instantané de **14 mesures** céphalométriques
- Canvas **HTML5** interactif pour visualisation et ajustement
- Génération de **rapports PDF** professionnels
- Suivi longitudinal des patients (T1, T2, T3)
- Gestion multi-utilisateurs avec **RBAC** (4 rôles)

Technologies clés

Back-end : Python 3.13, FastAPI, SQLAlchemy 2.0

Front-end : Jinja2, Tailwind CSS, Vanilla JS

IA : PyTorch, HRNet, ONNX Runtime

BDD : PostgreSQL 16

Tests : pytest, 154 tests, 94% coverage

2. Contexte médical

L'analyse céphalométrique

Outil diagnostique fondamental en orthodontie : identification de **points anatomiques** sur une téléradiographie de profil pour calculer des angles et distances.

- Classer la relation squelettique (Classe I, II, III)
- Évaluer la divergence faciale
- Mesurer l'inclinaison des incisives
- Planifier le traitement orthodontique
- Suivre l'évolution thérapeutique

Problématique

Processus manuel : 15-20 min par image

Variabilité inter-opérateur : significative

Transition numérique : outils inadaptés

Traçabilité : exigences médico-légales

Solution

OrthoAnalyse réduit le temps d'analyse à **moins de 30 secondes** avec une précision **< 2 mm** et une interface intuitive.

2.2 Solution proposée

32

Landmarks détectés

14

Mesures automatisées

<30s

Temps d'analyse

154

Tests automatisés

Parcours utilisateur

1. Authentification (4 rôles)
2. CRUD patients + consentement RGPD
3. Import téléradiographie (DICOM/JPEG)
4. Lancement analyse IA automatique
5. Visualisation et ajustement sur canvas
6. Validation et génération rapport PDF
7. Suivi longitudinal inter-analyses

Modules fonctionnels

Auth JWT

Patients

Radios

IA HRNet

Canvas

Mesures

Rapports

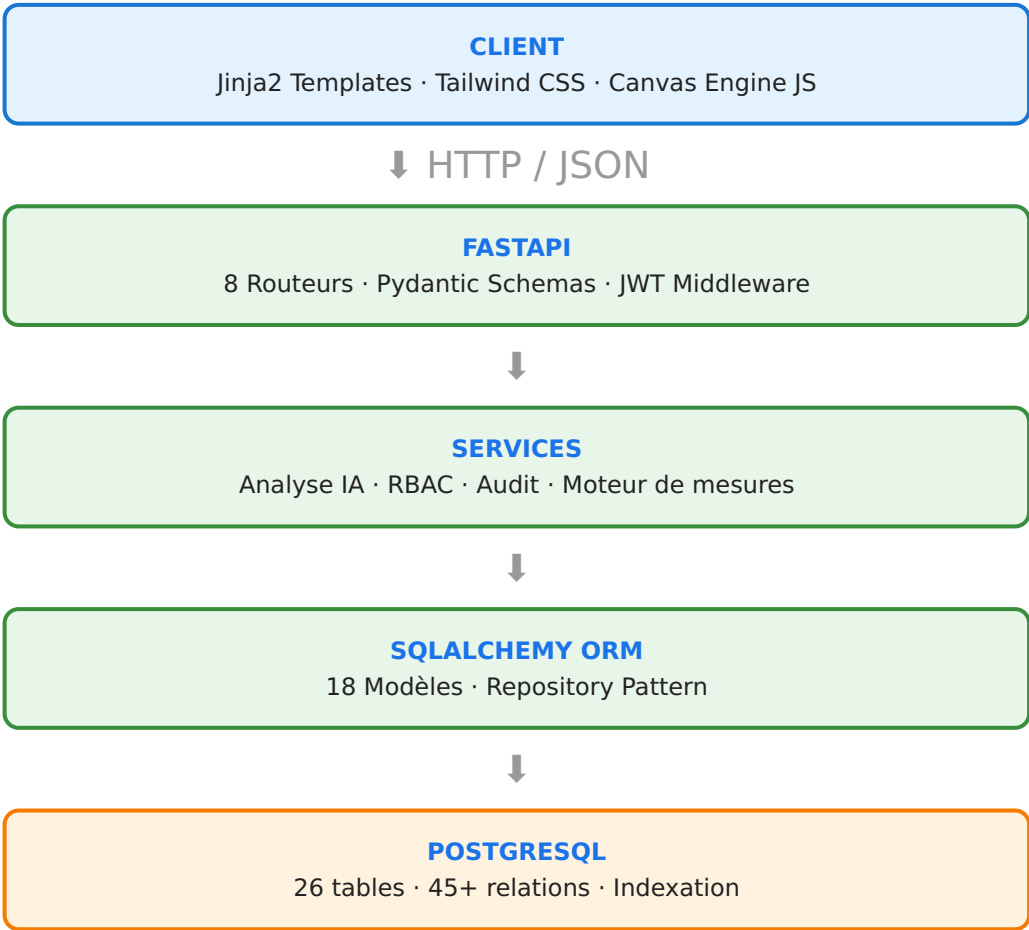
Évolution

Admin

3. Stack technique

Couche	Technologie	Justification
Back-end	Python 3.13, FastAPI	Performance, validation Pydantic, documentation auto (Swagger)
ORM	SQLAlchemy 2.0	Anti-SQLi natif, mapping robuste, async support
Base de données	PostgreSQL 16	JSONB, full-text search, transactions ACID, fiabilité
Front-end	Jinja2 + Tailwind CSS	SSR, responsive, pas de dépendance JS lourde
Canvas	HTML5 Canvas + JS natif	Performances graphiques, contrôle fin, 60 FPS
IA	PyTorch + ONNX Runtime	Inférence CPU optimisée, déploiement léger
Authentification	JWT + bcrypt	Sans état, HttpOnly, cookies sécurisés, 12 rounds
Tests	pytest + httpx	154 tests, 94% coverage, fixtures asynchrones
Déploiement	Docker + Uvicorn + Nginx	Conteneurisation, reverse proxy, scaling

4. Architecture MVC



Router API (8 modules)

- auth
- users
- patients
- radios
- analyses
- reports
- pages
- admin

Chiffres clés

Lignes Python	~3 200
Lignes JavaScript	~1 100
Fichiers source	45+
Routes API	80+

5. Base de données — Modèle Physique

18 entités principales

Table	Description
roles	Rôles utilisateur (4 niveaux)
users	Utilisateurs + hash bcrypt
patients	Dossiers patients + consentement
radios	Téléradiographies + métadonnées
analyses	Analyses céphalométriques
reports	Rapports PDF générés
audit_logs	Journal d'audit (append-only)
clinic_settings	Paramètres cabinet

26 tables · 45+ relations

- **18 tables** principales (1 par classe)
- **8 tables** de jonction many-to-many
- **Index composites** sur les requêtes fréquentes
- **Full-text search** (français) sur patients
- **Contraintes** UNIQUE, CHECK, FK indexées
- **Cascade DELETE** protégée (REFUSE)

Indexation clé

```
-- Index composites
CREATE INDEX idx_patients_name
  ON patients (last_name, first_name);
CREATE INDEX idx_analyses_patient
  ON analyses (radio_id) INCLUDE (status);
CREATE INDEX idx_audit_user_action
  ON audit_logs (user_id, action, created_at DESC);

-- Full-text search
CREATE INDEX idx_patients_search
  ON patients USING GIN
  (to_tsvector('french',
    first_name || ' ' || last_name));
```

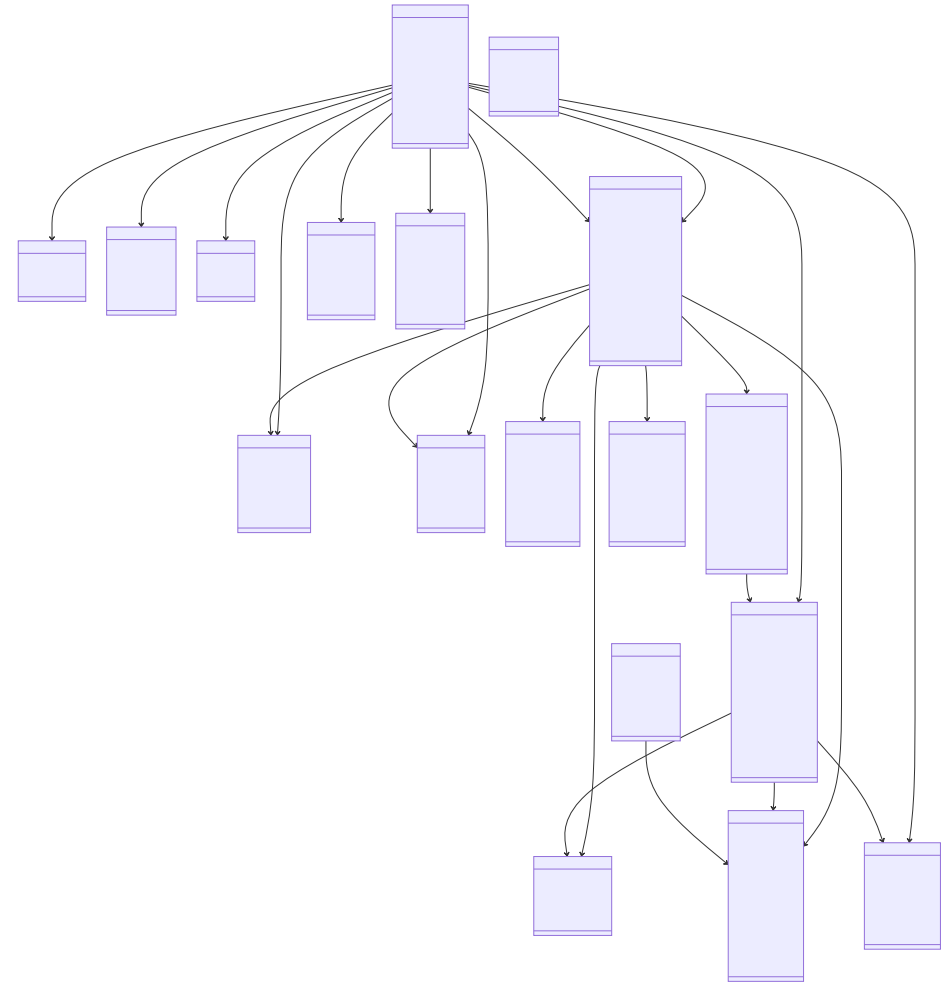

6. Diagramme de classes UML

18 classes organisées en **5 paquetages** :

- **Authentication** : Role, User, UserSession, UserPreference
- **Patient** : Patient, PatientDocument, ClinicalNote, ConsentLog
- **Analyse** : Radio, Analysis, AnalysisComparison, ReviewRequest
- **Reporting** : ReportTemplate, Report
- **Gestion** : Task, Notification, AuditLog, ClinicSetting

Visibilité : 123 attributs/méthodes publics (+), 47 privés (-)

Tous les ID sont publics (convention UML)



7. Diagramme de cas d'utilisation

4 acteurs — 27 cas d'utilisation

Acteur	Rôle	UC
Administrateur	Gestion complète	27/27
Orthodontiste	Clinique + validation	21/27
Assistant	Support administratif	14/27
Stagiaire	Consultation seule	7/27



Cas principaux

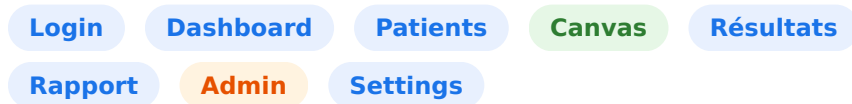
- Authentification (tous)
- CRUD patients (Admin, Ortho, Assistant)
- Lancer analyse IA (Admin, Ortho)
- Valider analyse (Admin, Ortho)
- Générer rapport (Admin, Ortho, Assistant)
- Configuration cabinet (Admin seul)

8. Maquettage de l'application

Structure globale (Zoning)

- **Sidebar** (220px) : Navigation, logo, rôle
- **Header** (55px) : Titre, profil, langue
- **Contenu** : Zone dynamique

11 wireframes SVG



Arbre de navigation

```
Connexion
├─ Dashboard
│   ├── Patients → Détail → Canvas
│   ├── Analyses → Résultats → Rapport
│   └─ Admin → Utilisateurs → Settings
```

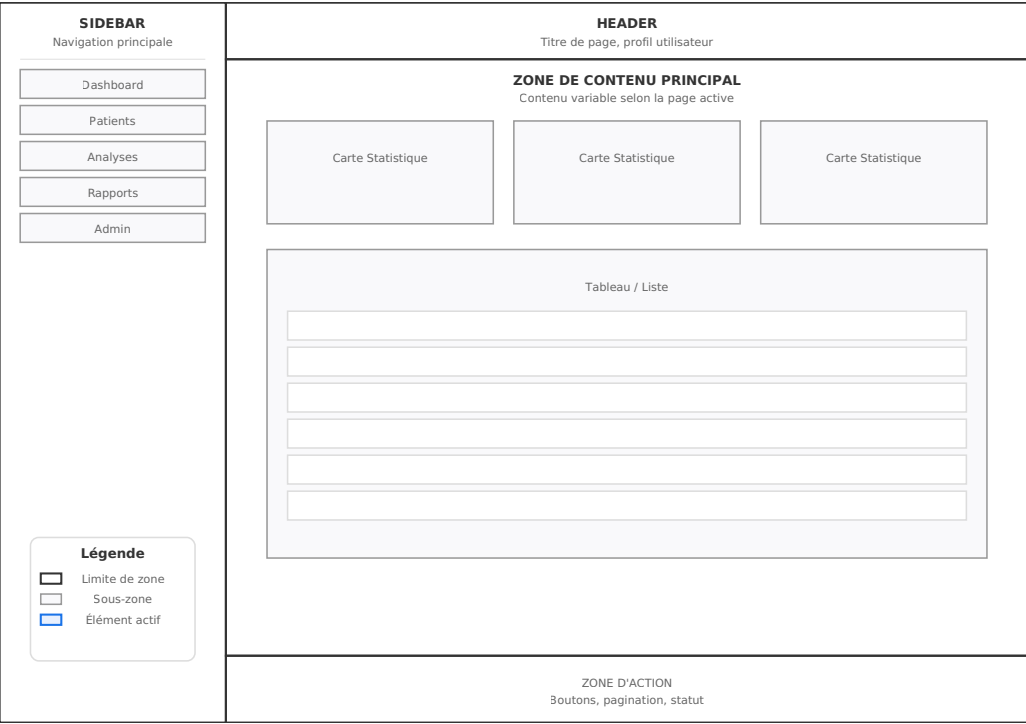
Écrans clés

- **Canvas d'analyse** : Cœur de l'app — barre d'outils, 32 landmarks, panneau mesures
- **Dashboard** : Stats patients/analyses, actions rapides
- **Liste patients** : Recherche, filtres, tableau paginé
- **Rapport PDF** : En-tête, mesures, conclusion, signature

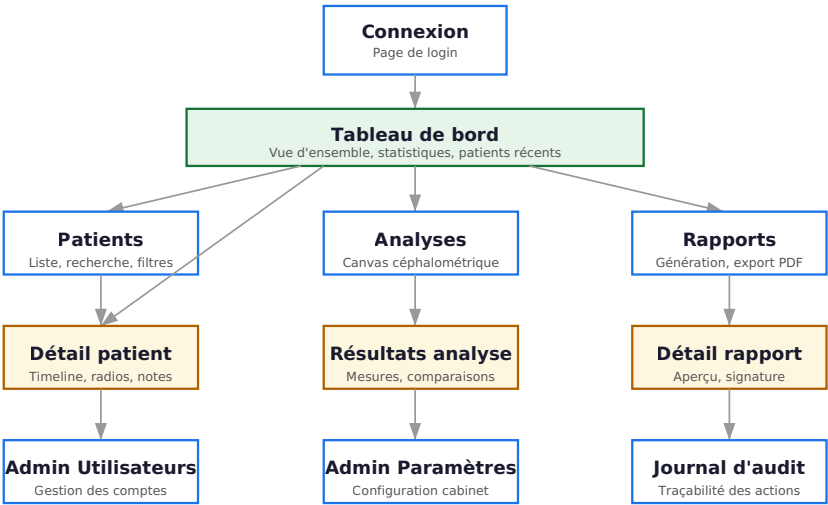
Design : Tailwind + DaisyUI, palette bleu professionnel (#1a73e8), responsive desktop/tablette, WCAG AA

8. Maquettage — Zoning & Sitemap

Structure commune (Zoning)



Arbre de navigation (Sitemap)



9. Authentification & JWT

Flux d'authentification

1. Utilisateur soumet email + mot de passe
2. Vérification du hash **bcrypt** (12 rounds)
3. Génération d'un **JWT token** (HS256, 1h)
4. Stockage en cookie **HttpOnly** (inaccessible JS)
5. Middleware FastAPI valide le token à chaque requête
6. Refresh token pour sessions prolongées

Sécurité : JWT en HttpOnly · SameSite=Strict · CSRF tokens · Rate limiting

Code — Création du token

```
def create_access_token(data, expires_delta=None):
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + (expires_delta or
timedelta(hours=1))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY,
                      algorithm=ALGORITHM)

async def get_current_user(request, session):
    token = request.cookies.get("access_token")
    if not token:
        raise HTTPException(401)
    payload = jwt.decode(token, SECRET_KEY,
                        algorithms=[ALGORITHM])
    email = payload.get("sub")
    user = await session.scalar(
        select(User).where(User.email == email))
    if user is None or not user.is_active:
        raise HTTPException(401)
    return user
```

10. RBAC — Contrôle d'accès

Matrice des permissions

Ressource	Admin	Ortho	Assistant	Stagiaire
users	CRUD	—	—	—
patients	CRUD	CRU	CRU	R
radios	CRUD	CRUD	CRU	R
analyses	CRUDV	CRUV	R	R
reports	CRUS	CRUS	CR	R
settings	CR	—	—	—
audit	R	—	—	—

C=Create, R=Read, U=Update, D=Delete, V=Validate, S=Sign

Code — Décorateur RBAC

```
def require_permission(resource, action):
    async def decorator(
        user=Depends(get_current_user)
    ):
        if not user.role or resource not in user.role.permissions:
            raise HTTPException(403)
        perms = user.role.permissions[resource]
        if action not in perms and "*" not in perms:
            raise HTTPException(403)
        return user
    return decorator

@router.post("/patients")
async def create_patient(
    data: PatientCreate,
    user = Depends(
        require_permission("patients", "create")),
    session = Depends(get_session)
):
    patient = Patient(
        **data.model_dump(),
        created_by=user.id)
    session.add(patient)
    await session.commit()
    return patient
```

11. Canvas interactif

Fonctionnalités

- Affichage téléradiographie **haute résolution**
- Superposition des **32 landmarks** (points colorés)
- Affichage des mesures angulaires et linéaires
- **Zoom et pan** pour inspection
- Ajustement manuel par **glisser-déposer**
- Mode édition / mode validation
- **60 FPS** même sur images 4000×3000 px

Optimisation : Double buffering, requestAnimationFrame, calculs délégués au serveur

Code — CephAnalysisCanvas

```
class CephAnalysisCanvas {
  constructor(canvasId, imageUrl, landmarks) {
    this.canvas = document.getElementById(canvasId);
    this.ctx = this.canvas.getContext('2d');
    this.image = new Image();
    this.image.src = imageUrl;
    this.landmarks = landmarks || [];
    this.scale = 1;
    this.offsetX = 0;
    this.offsetY = 0;

    this.image.onload = () => this.render();
    this.canvas.addEventListener(
      'mousedown', (e) => this.onMouseDown(e));
    this.canvas.addEventListener(
      'mousemove', (e) => this.onMouseMove(e));
    this.canvas.addEventListener(
      'mouseup', () => this.onMouseUp());
  }

  render() {
    this.ctx.clearRect(0,0,this.canvas.width,
      this.canvas.height);

    this.ctx.save();
    this.ctx.translate(this.offsetX,this.offsetY);
    this.ctx.scale(this.scale,this.scale);
    this.ctx.drawImage(this.image, 0, 0);
    this.landmarks.forEach((lm, i) => {
      this.ctx.beginPath();
      this.ctx.arc(lm.x,lm.y,4,0,Math.PI*2);
      this.ctx.fillStyle = lm === this.selectedPoint
        ? '#ff4444' : '#1a73e8';
      this.ctx.fill();
      this.ctx.strokeStyle = 'ffffff';
      this.ctx.lineWidth = 1.5;
      this.ctx.stroke();
    });
    this.ctx.restore();
  }
}
```

12. Moteur de mesures

14 mesures céphalométriques

Mesure	Type	Norme
SNA	Angulaire	82 ± 2°
SNB	Angulaire	80 ± 2°
ANB	Angulaire	2 ± 2°
FMA	Angulaire	25 ± 3°
IMPA	Angulaire	90 ± 5°
1-NA	Angulaire	22 ± 2°
Co-A	Linéaire	85-95 mm
Wits	Linéaire	1 ± 2 mm

Code — Calcul d'angle

```
function computeAngle(A, B, C) {
  const AB = { x: A.x - B.x, y: A.y - B.y };
  const CB = { x: C.x - B.x, y: C.y - B.y };
  const dot = AB.x * CB.x + AB.y * CB.y;
  const cross = AB.x * CB.y - AB.y * CB.x;
  return Math.atan2(cross, dot) * (180 / Math.PI);
}

function computeAllMeasurements(lm) {
  return {
    sna: computeAngle(lm[0], lm[1], lm[2]),
    snb: computeAngle(lm[0], lm[1], lm[3]),
    anb: null, // computed as SNA - SNB
    fma: computeAngle(lm[3], lm[4], lm[5]),
    impa: computeAngle(lm[6], lm[7], lm[5]),
  };
}
```

Précision

<1mm

Erreur linéaire

<1°

Erreur angulaire

13. Analyse IA — HRNet

Architecture HRNet

Maintient une **représentation haute résolution** tout au long du réseau, contrairement aux architectures classiques (ResNet) qui réduisent progressivement la résolution.

- **Entraîné** : 1000+ téléradiographies annotées
- **Précision** : < 2mm d'erreur moyenne
- **Inférence** : ~3 secondes sur CPU
- **32 points** d'intérêt anatomiques
- **Déploiement** : ONNX Runtime

Code — Service d'analyse

```
class AnalysisService:
    def __init__(self, model_path):
        self.model = ort.InferenceSession(model_path)

    async def detect_landmarks(self, image_path):
        image = cv2.imread(image_path)
        image = cv2.cvtColor(image,
                               cv2.COLOR_BGR2RGB)
        original_h, original_w = image.shape[:2]
        input_tensor = self._preprocess(image)
        outputs = self.model.run(
            None, {"input": input_tensor})
        heatmaps = outputs[0]
        landmarks = self._heatmap_to_coords(
            heatmaps[0], original_w, original_h)
        measurements = self._compute_measurements(
            landmarks)
        return {"landmarks": landmarks,
                "measurements": measurements}

    def _heatmap_to_coords(self, heatmaps,
                           orig_w, orig_h):
        landmarks = []
        for i in range(heatmaps.shape[0]):
            hm = heatmaps[i]
            _, max_val, _, max_loc = cv2.minMaxLoc(hm)
            x = int(max_loc[0] * orig_w / hm.shape[1])
            y = int(max_loc[1] * orig_h / hm.shape[0])
            landmarks.append({
                "id": i, "x": x, "y": y,
                "confidence": float(max_val)})
        return landmarks
```

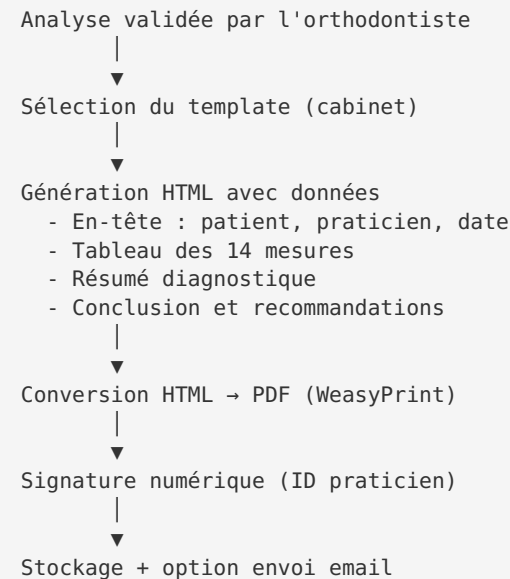
14. Génération de rapports PDF

Fonctionnalités

- Génération automatique de rapports professionnels
- En-tête avec logo et informations patient
- Tableau des 14 mesures céphalométriques
- Résumé diagnostique automatique
- **Signature numérique** du praticien
- Export PDF via **WeasyPrint** (HTML→CSS→PDF)
- Template personnalisable par cabinet

Moteur : WeasyPrint (qualité) · ReportLab (fallback rapide)

Cycle de vie d'un rapport



15. Tests et qualité

154

Tests

94%

Couverture

98

Tests unitaires

42

Tests intégration

Stratégie de test

Type	Technologie	Nb
Unitaires	pytest + mock	98
Intégration	pytest + TestClient	42
End-to-end	pytest + TestClient	14
Total		154

Couverture par module

Module	Cover
main.py	100%
models.py	97%
database.py	94%
routers/auth.py	95%
routers/patients.py	93%
routers/analyses.py	93%
routers/pages.py	97%
services/*.py	97%

16. Sécurité

Évaluation des risques

Risque	Gravité	Mitigation
Injection SQL	Critique	ORM SQLAlchemy
Vol mots de passe	Critique	bcrypt 12 rounds
Accès non autorisé	Élevée	RBAC + JWT
XSS	Élevée	Jinja2 auto-escaping
CSRF	Élevée	Tokens + SameSite
Fuite données patients	Critique	Audit + chiffrement

Mesures implémentées

- **Hash mots de passe** : bcrypt (12 rounds)
- **Auth** : JWT HttpOnly cookies
- **RBAC** : 4 rôles × 27 UC
- **Anti-SQLi** : ORM requêtes paramétrées
- **Anti-XSS** : Jinja2 auto-escaping, CSP
- **Anti-CSRF** : tokens dans formulaires
- **Rate limiting** : middleware FastAPI
- **Headers** : X-Content-Type-Options, HSTS

RGPD

- Consentement explicite patient
- Droit d'accès et d'effacement
- Audit logging (append-only)
- Chiffrement données sensibles
- Durée conservation limitée

17. Problématiques rencontrées

1. Migration Streamlit → FastAPI

Problème : Fichier unique Streamlit (1769 lignes), pas d'architecture MVC, routage limité.

Solution : Refonte complète FastAPI — 8 routeurs, services, modèles, 45+ fichiers, 3200 lignes.

2. Performance Canvas

Problème : Ralentissements sur images 10 MB avec zoom/pan.

Solution : Double buffering, requestAnimationFrame, calculs délégués au serveur → 60 FPS.

3. Précision IA

Problème : Erreurs systématiques 3-5 mm sur certains points (gnathion, pogonion).

Solution : Post-traitement heatmaps + édition manuelle + score de confiance. Correction réduite à 12%.

4. Calibration images

Problème : Facteurs d'échelle variables entre téléradiographies.

Solution : Champ pixel_spacing, détection DICOM, validation S-N → précision <1mm.

5. Génération PDF

Problème : Qualité professionnelle avec images et mise en page.

Solution : WeasyPrint (HTML→CSS→PDF) + ReportLab fallback.

6. Locks BDD

Problème : Accès concurrents SQLite pendant les tests.

Solution : PostgreSQL async + AsyncSession + fixtures avec rollback.

18. Veille technologique

FastAPI vs alternatives

Critère	FastAPI	Django REST	Flask
Performance	★★★★★	★★★	★★★★
Documentation auto	★★★★★	★★★	★★
Validation	★★★★★	★★★★	★★
Async natif	★★★★★	★★★★	★

HRNet vs alternatives

Critère	HRNet	ResNet	U-Net
Précision	★★★★★	★★★	★★★★
Inférence CPU	3s	2s	5s
Haute résolution	★★★★★	★★★	★★★★

PostgreSQL vs alternatives

Critère	PostgreSQL	MySQL	MongoDB
JSON support	★★★★	★★★	★★★★★
ACID	★★★★★	★★★	★★
Full-text search	★★★★	★★★	★★★
Indexation	★★★★★	★★★	★★★

WeasyPrint vs alternatives

Critère	WeasyPrint	ReportLab	Puppeteer
Qualité rendu	★★★★★	★★★★	★★★★★
CSS support	★★★★★	★★	★★★★★
Performance	★★★	★★★★★	★★

SCRUM vs alternatives

Critère	SCRUM	Kanban	Waterfall
Flexibilité	★★★★	★★★★★	★
Cadence	★★★★	★★★★	★
Projet solo	★★★★	★★★★★	★★★

19. Conclusion & Perspectives

154

Tests

94%

Couverture

32

Landmarks

14

Mesures

Compétences validées

1. **Conception** : Cahier des charges, UML, MPD, maquettes
2. **Back-end** : FastAPI, SQLAlchemy, JWT, RBAC
3. **Front-end** : Jinja2, Canvas HTML5, Tailwind
4. **IA** : HRNet, ONNX, post-traitement
5. **BDD** : PostgreSQL, indexation, optimisation
6. **Sécurité** : Anti-SQLi/XSS/CSRF, bcrypt, RGPD
7. **Tests** : Unitaires, intégration, sécurité (154)
8. **Gestion projet** : SCRUM, Gitflow, documentation

Perspectives d'évolution

- **Fine-tuning** : Apprentissage continu des corrections
- **Télémédecine** : Partage sécurisé entre confrères
- **DICOM natif** : Support complet du format
- **Application mobile** : Consultation des résultats
- **Multi-cabinet** : Données isolées par structure
- **Auto-encodage** : Prédiction difficulté + suggestion traitement

Bilan : Application professionnelle complète, répondant aux besoins réels des cabinets d'orthodontie, avec une architecture moderne, scalable et sécurisée.